

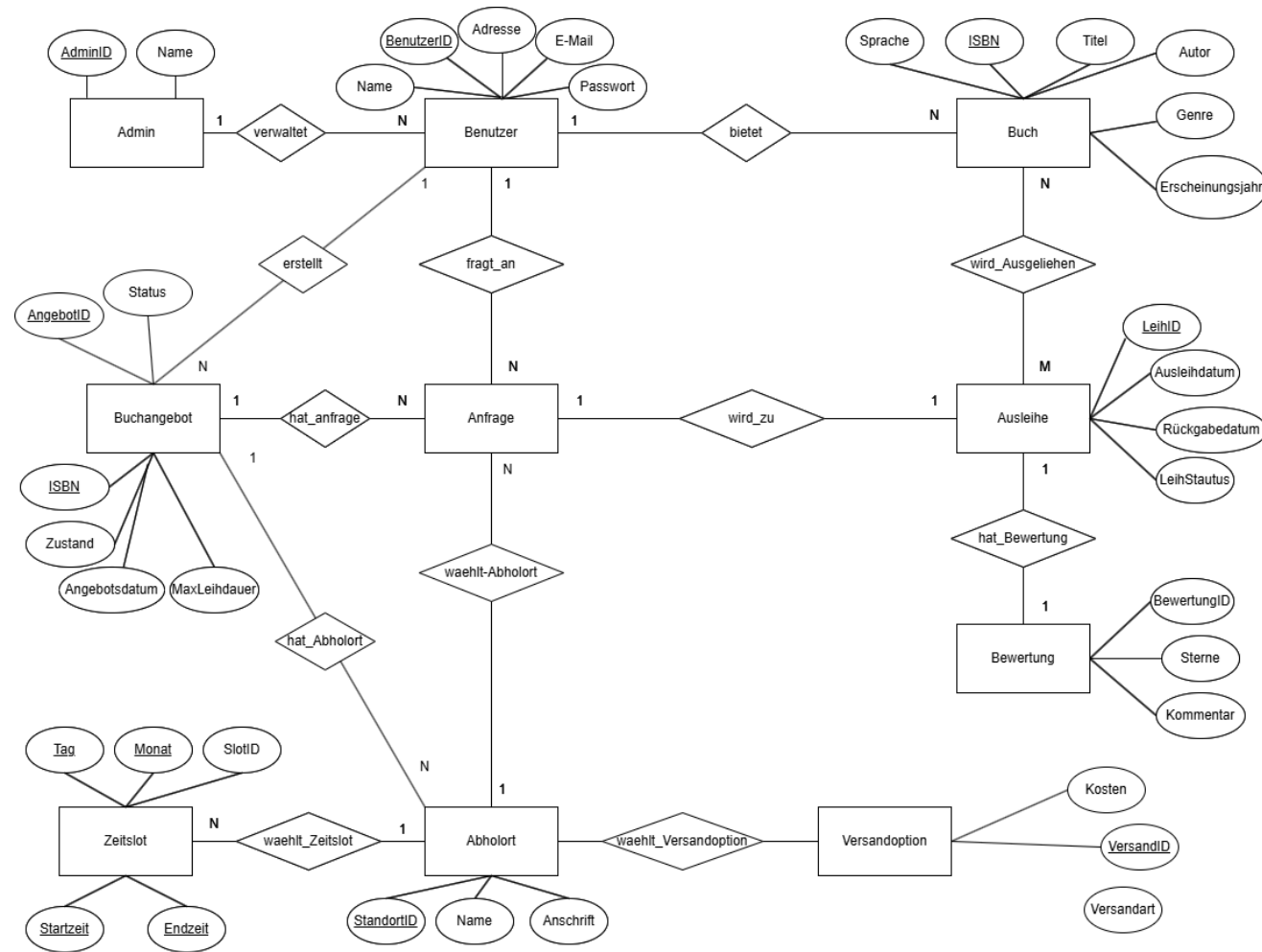
Datenbank für eine Buchtausch-App

- Sandro Amberger
- Matrikelnummer: [IU14109864]
- Kurs: DLBDSPBDM01D
- IU Internationale Hochschule
- 23.04.2026

Zielsetzung der Arbeit

- Konzeption und Implementierung eines relationalen Datenbankschemas für eine Buchtausch-App.
- Speicherung, Verknüpfung und Abfrage aller relevanten Daten zu Benutzern, Büchern und Ausleihvorgängen.
- Umsetzung in einem SQL-basierten Datenbankmanagementsystem.
- Verwendung von Dummy-Daten zur Prüfung der Funktionalität und Darstellung der Ergebnisse.

Entity-Relationship-Modell



Benutzer

- Die Entität **Benutzer** speichert die registrierten Personen der Plattform.
- Wichtige Attribute sind BenutzerID, Name, Adresse, E-Mail und Passwort.
- Primärschlüssel: BenutzerID.
- Ein Beispieldatensatz wurde mit `INSERT INTO Benutzer (...) VALUES (...);` eingefügt und anschließend mit `SELECT * FROM Benutzer WHERE BenutzerID = ...;` überprüft.

```
1 INSERT INTO Benutzer (BenutzerID, Name, Adresse, EMail, Passwort)
2 VALUES (11, 'Sandro Amberger', 'Graz, Österreich', 'sandro@example.com', '123456');
```

```
Ausführung wurde ohne Fehler beendet.
Ergebnis: Abfrage erfolgreich ausgeführt. Benötigte 2 ms , 1 Zeilen betroffen
In Zeile 1:
INSERT INTO Benutzer (BenutzerID, Name, Adresse, EMail, Passwort)
VALUES (11, 'Sandro Amberger', 'Graz, Österreich', 'sandro@example.com', '123456');
```

SQL 1*

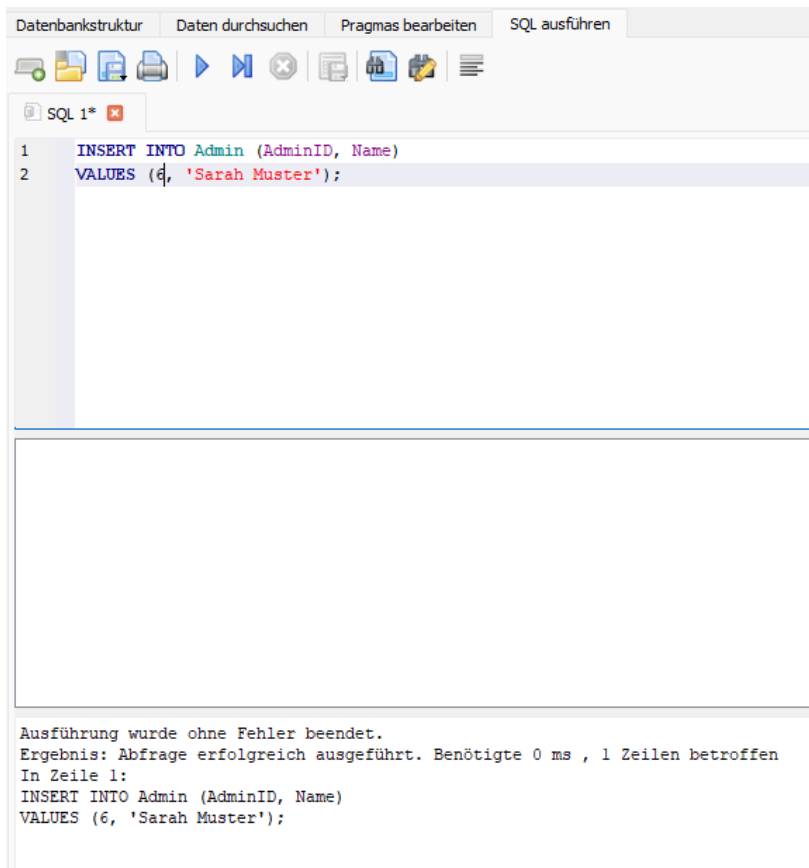
```
1 SELECT *
2 FROM Benutzer
3 WHERE BenutzerID = 11;
```

	BenutzerID	Name	Adresse	Email	Passwort
1	11	Sandro Amberger	Graz, Österreich	sandro@example.com	123456

```
Ausführung wurde ohne Fehler beendet.
Ergebnis: 1 Zeilen in 12 ms zurückgegeben
In Zeile 1:
SELECT *
FROM Benutzer
WHERE BenutzerID = 11;
```

Admin

- Die Entität **Admin** dient der Pflege und Kontrolle der Datenbestände.
- Wichtige Attribute sind AdminID und Name.
- Primärschlüssel: AdminID.
- Ein Beispieldatensatz wurde mit INSERT INTO Admin (...) VALUES (...); eingefügt und mit SELECT * FROM Admin WHERE AdminID = ...; im DBMS geprüft.

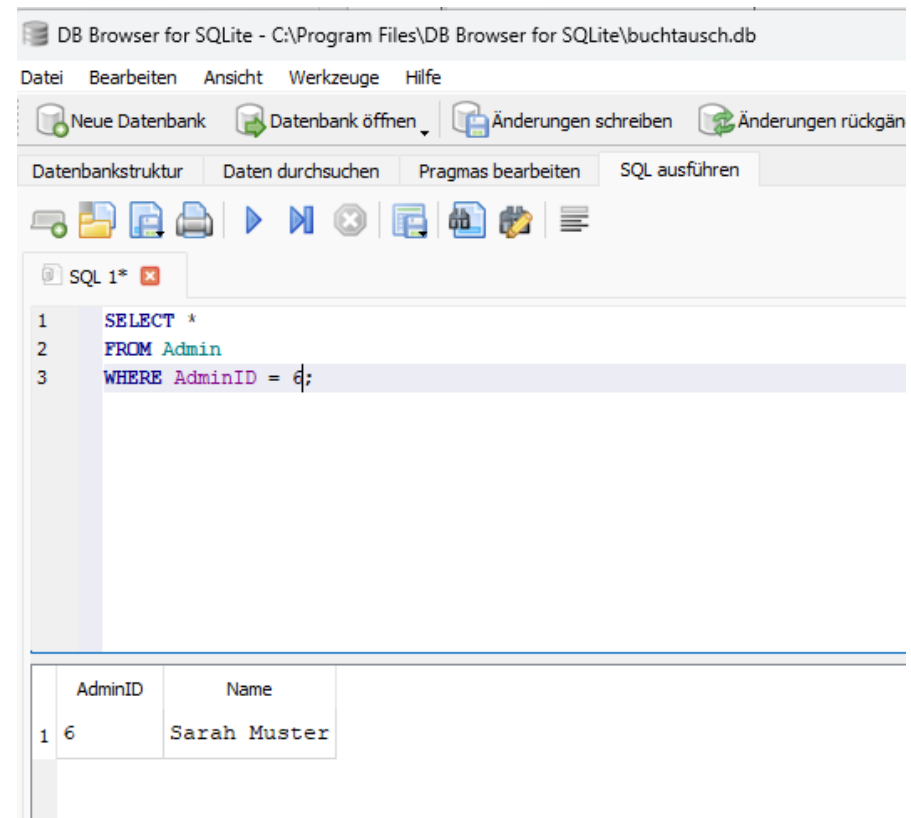


The screenshot shows a SQL editor window with a menu bar (Datenbankstruktur, Daten durchsuchen, Pragmas bearbeiten, SQL ausführen) and a toolbar. The SQL editor contains the following code:

```
1 INSERT INTO Admin (AdminID, Name)
2 VALUES (6, 'Sarah Muster');
```

Below the editor, a status bar indicates the execution was successful:

Ausführung wurde ohne Fehler beendet.
Ergebnis: Abfrage erfolgreich ausgeführt. Benötigte 0 ms , 1 Zeilen betroffen
In Zeile 1:
INSERT INTO Admin (AdminID, Name)
VALUES (6, 'Sarah Muster');



The screenshot shows a SQL editor window with a menu bar (Datei, Bearbeiten, Ansicht, Werkzeuge, Hilfe) and a toolbar. The SQL editor contains the following code:

```
1 SELECT *
2 FROM Admin
3 WHERE AdminID = 6;
```

Below the editor, a table displays the result of the query:

	AdminID	Name
1	6	Sarah Muster

Buch

- Die Entität **Buch** enthält die bibliografischen Stammdaten der Bücher.
- Wichtige Attribute sind BuchID, Titel, Autor, Sprache, ISBN, Genre und Erscheinungsjahr.
- Primärschlüssel: BuchID.
- Die Trennung von Buch und Buchangebot vermeidet Redundanzen. Ein Datensatz wurde per INSERT INTO Buch (...) VALUES (...); eingefügt und danach mit SELECT überprüft.

```
1 INSERT INTO Buch (BuchID, Titel, Autor, Sprache, ISBN, Genre, Erscheinungsjahr)
2 VALUES (11, 'Der Alchimist', 'Paulo Coelho', 'Deutsch', '9783257229533', 'Roman', 1988);
```

```
Ausführung wurde ohne Fehler beendet.
Ergebnis: Abfrage erfolgreich ausgeführt. Benötigte 0 ms , 1 Zeilen betroffen
In Zeile 1:
INSERT INTO Buch (BuchID, Titel, Autor, Sprache, ISBN, Genre, Erscheinungsjahr)
VALUES (11, 'Der Alchimist', 'Paulo Coelho', 'Deutsch', '9783257229533', 'Roman', 1988);
```

Datenbankstruktur Daten durchsuchen Pragmas bearbeiten SQL ausführen

SQL 1*

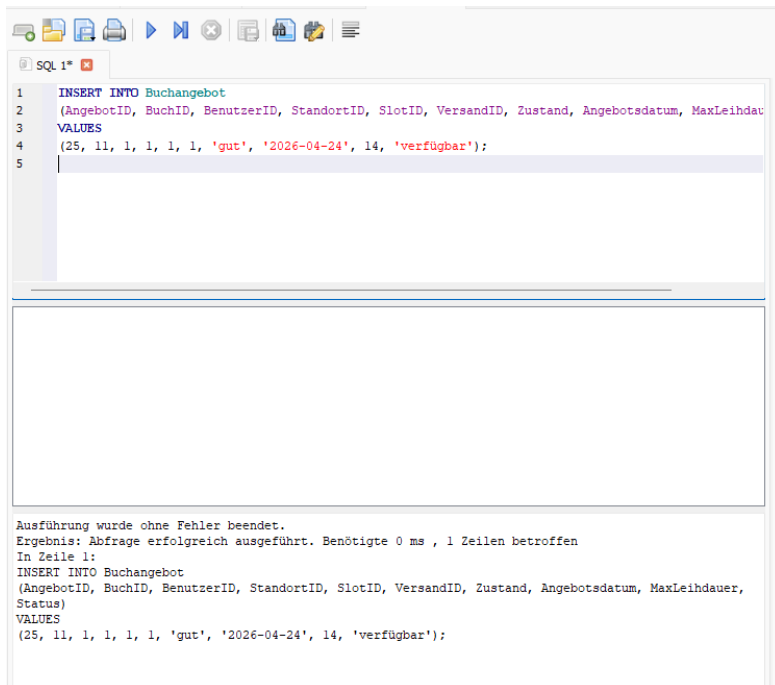
```
1 SELECT *
2 FROM Buch
3 WHERE BuchID = 11;
```

	BuchID	Titel	Autor	Sprache	ISBN	Genre	Erscheinungsjahr
1	11	Der Alchimist	Paulo Coelho	Deutsch	9783257229533	Roman	1988

Ausführung wurde ohne Fehler beendet.
Ergebnis: 1 Zeilen in 6 ms zurückgegeben
In Zeile 1:
SELECT *
FROM Buch
WHERE BuchID = 11;

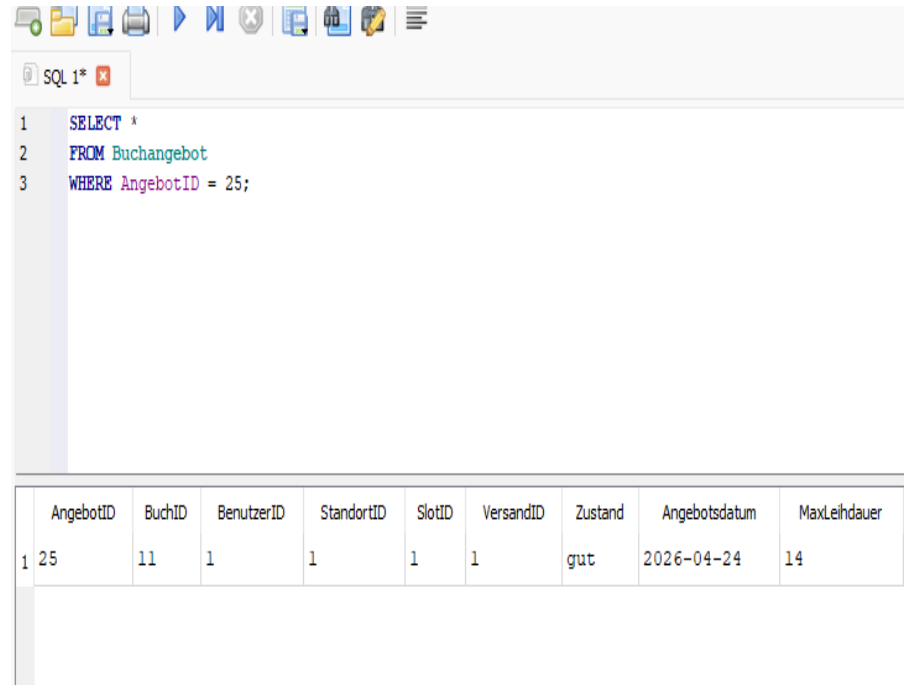
Buchangebot

- Die Entität **Buchangebot** beschreibt ein konkretes Angebot eines Buches innerhalb der App.
- Wichtige Attribute sind AngebotID, BuchID, BenutzerID, StandortID, SlotID, VersandID, Zustand, Angebotsdatum, MaxLeihdauer und Status.
- Primärschlüssel: AngebotID.
- Ein Beispieldatensatz wurde mit INSERT INTO Buchangebot (...) VALUES (...); eingefügt und anschließend mit SELECT * FROM Buchangebot WHERE AngebotID = ...; überprüft.



```
1 INSERT INTO Buchangebot
2 (AngebotID, BuchID, BenutzerID, StandortID, SlotID, VersandID, Zustand, Angebotsdatum, MaxLeihdauer,
3 Status)
4 VALUES
5 (25, 11, 1, 1, 1, 1, 'gut', '2026-04-24', 14, 'verfügbar');
```

Ausführung wurde ohne Fehler beendet.
Ergebnis: Abfrage erfolgreich ausgeführt. Benötigte 0 ms , 1 Zeilen betroffen
In Zeile 1:
INSERT INTO Buchangebot
(AngebotID, BuchID, BenutzerID, StandortID, SlotID, VersandID, Zustand, Angebotsdatum, MaxLeihdauer, Status)
VALUES
(25, 11, 1, 1, 1, 1, 'gut', '2026-04-24', 14, 'verfügbar');



```
1 SELECT *
2 FROM Buchangebot
3 WHERE AngebotID = 25;
```

	AngebotID	BuchID	BenutzerID	StandortID	SlotID	VersandID	Zustand	Angebotsdatum	MaxLeihdauer
1	25	11	1	1	1	1	gut	2026-04-24	14

Anfrage

- Die Entität **Anfrage** speichert Rückfragen und Leihanfragen zu einem Buchangebot.
- Wichtige Attribute sind AnfrageID, AngebotID, BenutzerID und Status.
- Primärschlüssel: AnfrageID.
- Die Tabelle dokumentiert Anfragen von Benutzern zu vorhandenen Angeboten. Ein Datensatz wurde per INSERT INTO Anfrage (...) VALUES (...); eingefügt und anschließend mit SELECT kontrolliert.

```
1 INSERT INTO Buchangebot
2 (AngebotID, BuchID, BenutzerID, StandortID, SlotID, VersandID, Zustand,
3 VALUES
4 (25, 11, 1, 1, 1, 1, 'gut', '2026-04-24', 14, 'verfügbar');
5
```

Ausführung wurde ohne Fehler beendet.
Ergebnis: Abfrage erfolgreich ausgeführt. Benötigte 0 ms , 1 Zeilen betroffen
In Zeile 1:
INSERT INTO Buchangebot
(AngebotID, BuchID, BenutzerID, StandortID, SlotID, VersandID, Zustand, Angebotsdatum, MaxLeihdauer, Status)
VALUES
(25, 11, 1, 1, 1, 1, 'gut', '2026-04-24', 14, 'verfügbar');

```
1 SELECT *
2 FROM Buchangebot
3 WHERE AngebotID = 25;
```

AngebotID	BuchID	BenutzerID	StandortID	SlotID	VersandID	Zustand	An
1 25	11	1	1	1	1	gut	2026

Ausführung wurde ohne Fehler beendet.
Ergebnis: 1 Zeilen in 4 ms zurückgegeben
In Zeile 1:
SELECT *
FROM Buchangebot
WHERE AngebotID = 25;

Ausleihe

- Die Entität **Ausleihe** dokumentiert bestätigte Leihvorgänge.
- Wichtige Attribute sind LeihID, AngebotID, AusleihenderID, Ausleihdatum, Rückgabedatum und LeihStatus.
- Primärschlüssel: LeihID.
- Ein Datensatz wurde mit INSERT INTO Ausleihe (...) VALUES (...); gespeichert und danach mit SELECT * FROM Ausleihe WHERE LeihID = ...; geprüft.

```
SQL 1*
1 INSERT INTO Ausleihe
2 (LeihID, AngebotID, AusleihenderID, Ausleihdatum, Rueckgabedatum, LeihStatus)
3 VALUES
4 (10, 25, 1, '2026-04-24', '2026-05-08', 'aktiv');
```

Ausführung wurde ohne Fehler beendet.
Ergebnis: Abfrage erfolgreich ausgeführt. Benötigte 0 ms , 1 Zeilen betroffen
In Zeile 1:
INSERT INTO Ausleihe
(LeihID, AngebotID, AusleihenderID, Ausleihdatum, Rueckgabedatum, LeihStatus)
VALUES
(10, 25, 1, '2026-04-24', '2026-05-08', 'aktiv');

Ausleihe einfügen.sql*

```
1 SELECT *
2 FROM Ausleihe
3 WHERE LeihID = 10;
```

	LeihID	AngebotID	AusleihenderID	Ausleihdatum	Rueckgabedatum	LeihStatus
1	10	25	1	2026-04-24	2026-05-08	aktiv

Ausführung wurde ohne Fehler beendet.
Ergebnis: 1 Zeilen in 4 ms zurückgegeben
In Zeile 1:
SELECT *
FROM Ausleihe
WHERE LeihID = 10;

Bewertung

- Die Entität **Bewertung** speichert Bewertungen nach abgeschlossener Ausleihe.
- Wichtige Attribute sind BewertungID, LeihID, BenutzerID, Sterne und Kommentar.
- Primärschlüssel: BewertungID.
- Ein Beispieldatensatz wurde per INSERT INTO Bewertung (...) VALUES (...); eingefügt und mit SELECT * FROM Bewertung WHERE BewertungID = ...; im DBMS kontrolliert.

```
1 INSERT INTO Bewertung
2 (BewertungID, LeihID, BenutzerID, Sterne, Kommentar, Bewertungsdatum)
3 VALUES
4 (10, 2, 1, 5, 'Die Ausleihe hat problemlos funktioniert.', '2026-04-24');
```

Ausführung wurde ohne Fehler beendet.
Ergebnis: Abfrage erfolgreich ausgeführt. Benötigte 0 ms , 1 Zeilen betroffen
In Zeile 1:
INSERT INTO Bewertung
(BewertungID, LeihID, BenutzerID, Sterne, Kommentar, Bewertungsdatum)
VALUES
(10, 2, 1, 5, 'Die Ausleihe hat problemlos funktioniert.', '2026-04-24');

```
1 SELECT *
2 FROM Bewertung
3 WHERE BewertungID = 10;
```

	BewertungID	LeihID	BenutzerID	Sterne	Kommentar	Bewertungsdatum
1	10	2	1	5	Die Ausleihe hat problemlos ...	2026-04-24

Ausführung wurde ohne Fehler beendet.
Ergebnis: 1 Zeilen in 4 ms zurückgegeben
In Zeile 1:
SELECT *
FROM Bewertung
WHERE BewertungID = 10;

Abholort

- Die Entität **Abholort** beschreibt Orte für die Übergabe eines Buches.
- Wichtige Attribute sind StandortID, Name und Anschrift.
- Primärschlüssel: StandortID.
- Ein Beispieldatensatz wurde mit INSERT INTO Abholort (...) VALUES (...); eingefügt und anschließend per SELECT * FROM Abholort WHERE StandortID = ...; überprüft.

```
1 INSERT INTO Abholort (StandortID, Name, Anschrift)
2 VALUES (20, 'Graz Mitte', 'Hauptplatz 1, Graz');
```

Ausführung wurde ohne Fehler beendet.
Ergebnis: Abfrage erfolgreich ausgeführt. Benötigte 0 ms , 1 Zeilen betroffen
In Zeile 1:
INSERT INTO Abholort (StandortID, Name, Anschrift)
VALUES (20, 'Graz Mitte', 'Hauptplatz 1, Graz');

```
abholort abfragen.sql
1 SELECT *
2 FROM Abholort
3 WHERE StandortID = 20;
```

	StandortID	Name	Anschrift
1	20	Graz Mitte	Hauptplatz 1, Graz

Ausführung wurde ohne Fehler beendet.
Ergebnis: 1 Zeilen in 4 ms zurückgegeben
In Zeile 1:
SELECT *
FROM Abholort
WHERE StandortID = 20;

Zeitslot

- Die Entität **Zeitslot** erfasst mögliche Übergabezeiten.
- Wichtige Attribute sind SlotID, Tag, Monat, Startzeit und Endzeit.
- Primärschlüssel: SlotID.
- Ein Datensatz wurde mit INSERT INTO Zeitslot (...) VALUES (...); eingefügt und danach mit SELECT * FROM Zeitslot WHERE SlotID = ...; kontrolliert.

```
SQL 1*
1 INSERT INTO Zeitslot (SlotID, Tag, Monat, Startzeit, Endzeit)
2 VALUES (11, 'Montag', 'April', '14:00', '16:00');
```

Ausführung wurde ohne Fehler beendet.
Ergebnis: Abfrage erfolgreich ausgeführt. Benötigte 0 ms , 1 Zeilen betroffen
In Zeile 1:
INSERT INTO Zeitslot (SlotID, Tag, Monat, Startzeit, Endzeit)
VALUES (11, 'Montag', 'April', '14:00', '16:00');

```
1 SELECT *
2 FROM Zeitslot
3 WHERE SlotID = 11;
```

	SlotID	Tag	Monat	Startzeit	Endzeit
1	11	Montag	April	14:00	16:00

Ausführung wurde ohne Fehler beendet.
Ergebnis: 1 Zeilen in 4 ms zurückgegeben
In Zeile 1:
SELECT *
FROM Zeitslot
WHERE SlotID = 11;

Versandoption

- Die Entität **Versandoption** speichert mögliche Versandarten für Buchangebote.
- Wichtige Attribute sind VersandID, Versandart und Kosten.
- Primärschlüssel: VersandID.
- Ein Beispieldatensatz wurde mit INSERT INTO Versandoption (...) VALUES (...); eingefügt und mit SELECT * FROM Versandoption WHERE VersandID = ...; überprüft.

```
SQL 1*  
1 INSERT INTO Versandoption (VersandID, Versandart, Kosten)  
2 VALUES (11, 'Post', 4.90);
```

Ausführung wurde ohne Fehler beendet.
Ergebnis: Abfrage erfolgreich ausgeführt. Benötigte 0 ms , 1 Zeilen betroffen
In Zeile 1:
INSERT INTO Versandoption (VersandID, Versandart, Kosten)
VALUES (11, 'Post', 4.90);

```
SQL 1*  
1 SELECT *  
2 FROM Versandoption  
3 WHERE VersandID = 11;
```

	VersandID	Versandart	Kosten
1	11	Post	4.9

Ausführung wurde ohne Fehler beendet.
Ergebnis: 1 Zeilen in 3 ms zurückgegeben
In Zeile 1:
SELECT *
FROM Versandoption
WHERE VersandID = 11;

Testfall mit Ergebnis

- Für Phase 2 ist mindestens ein Testfall erforderlich.
- Der Testfall prüft die Verknüpfung mehrerer Tabellen über JOINS.
- Angezeigt werden verfügbare Buchangebote mit Buchtitel, Benutzername, Zustand und Status.
- Die Abfrage liefert 10 Datensätze und wurde im DBMS erfolgreich ausgeführt.
- Der Screenshot dokumentiert SQL-Statement und Ergebnis.

```
1 SELECT
2     ba.AngebotID,
3     b.Titel,
4     u.Name AS Benutzername,
5     ba.Zustand,
6     ba.Status
7 FROM Buchangebot ba
8 JOIN Buch b ON ba.BuchID = b.BuchID
9 JOIN Benutzer u ON ba.BenutzerID = u.BenutzerID
10 WHERE ba.Status = 'verfügbar';
```

	AngebotID	Titel	Benutzername	Zustand	Status
1	1	Der Alchimist	Anna Weber	Sehr gut	verfügbar
2	2	Harry Potter	Max Hofer	Gut	verfügbar
3	3	Die Verwandlung	Sara Bauer	Neu	verfügbar
4	5	Sapiens	Nina Fuchs	Sehr gut	verfügbar
5	6	Momo	Paul Steiner	Gut	verfügbar
6	7	Das Parfum	Tina Wolf	Akzeptabel	verfügbar
7	8	Clean Code	David Kurz	Sehr gut	verfügbar
8	9	The Hobbit	Lea Moser	Neu	verfügbar
9	10	Brief an den Vater	Jonas Huber	Gut	verfügbar
10	25	Der Alchimist	Anna Weber	gut	verfügbar

Ausführung wurde ohne Fehler beendet.
Ergebnis: 10 Zeilen in 8 ms zurückgegeben
In Zeile 1:
SELECT
 ba.AngebotID,
 b.Titel,
 u.Name AS Benutzername,
 ba.Zustand,
 ba.Status
FROM Buchangebot ba
JOIN Buch b ON ba.BuchID = b.BuchID
JOIN Benutzer u ON ba.BenutzerID = u.BenutzerID
WHERE ba.Status = 'verfügbar';

Zusammenfassung der Implementierung

Im Rahmen der Implementierung wurde das in Phase 1 entwickelte ER-Modell in ein relationales Datenbankschema für die Buchtauschplattform überführt. Dabei wurden die zentralen Entitäten Benutzer, Admin, Buch, Buchangebot, Anfrage, Ausleihe, Bewertung, Abholort, Zeit Slot und Versandoption als eigenständige Tabellen umgesetzt. Für jede Tabelle wurden passende Attribute, Primärschlüssel und Fremdschlüssel definiert, damit die Beziehungen zwischen den Entitäten korrekt abgebildet werden.

Ein besonderer Schwerpunkt lag auf der Verknüpfung der Tabellen, damit typische Abläufe der Anwendung technisch unterstützt werden. Dazu gehören das Anlegen eines Buchangebots, das Stellen einer Anfrage, die Durchführung einer Ausleihe und die anschließende Bewertung. Die Tabellenstruktur wurde so aufgebaut, dass diese Prozesse logisch miteinander verbunden sind und die Daten konsistent gespeichert werden können.

Die Umsetzung erfolgte mithilfe von SQL-Statements zur Definition der Tabellen und zur Befüllung mit Testdaten. Anschließend wurden Testabfragen im Datenbankmanagementsystem ausgeführt, um die Funktionsfähigkeit der Datenbank zu überprüfen. Besonders wichtig war dabei, dass Verknüpfungen zwischen mehreren Tabellen korrekt funktionieren und die Ergebnisse nachvollziehbar dargestellt werden können.

Zusätzlich wurde ein Join-Testfall durchgeführt, um die Zusammenarbeit mehrerer Tabellen nachzuweisen. So konnten verfügbare Buchangebote gemeinsam mit dem jeweiligen Buchtitel und dem zugehörigen Benutzernamen angezeigt werden. Die Testergebnisse wurden per Screenshot dokumentiert und zeigen, dass das Datenbankschema korrekt aufgebaut ist und die grundlegenden Funktionen der Buchtauschplattform zuverlässig unterstützt.